

ANALYZING THE DIFFERENCE IN PERFORMANCE BETWEEN Q LEARNING AND SARSA ON GOLD EXPLORER PUZZLE

by

Mohamed Affan Dhankwala

A report submitted to Johns Hopkins University in conformity with the requirements for
705.741.8VL.SP25 Reinforcement Learning

Baltimore, Maryland

March 2025

ABSTRACT

Reinforcement learning allows agents to mathematically map out environments in effort to calculate the optimal path through them. This study evaluates the performance of both Q learning and SARSA algorithms in a controlled environment to assess their convergence rates, stability and overall performance. Our findings indicate that while both q learning and SARSA demonstrate capability to navigate the environment efficiently, there is no substantial evidence to support that one performs at a higher level than the other within this environment. SARSA also was much more susceptible to extremely high-error outliers which incited the need to measure performance of the two models via both mean and median. This behavior is linked to its on-policy nature where the agent becomes stuck within a cycle of states due to the closeness of their q values. In contrast, q learning, being an off-policy algorithm, updates the q table with the assumption that an agent will traverse the highest rewarding action eventually. This trend also existed when performing Q Learning and SARSA while exploring starts. The performance of the agents when training while exploring starts was slightly less than the when not exploring starts and this is due to those agents not converging quickly enough.

INTRODUCTION

Reinforcement learning (RL) allows models to navigate unknown environments with only a set of legal actions. To learn their environment, these agents create policies to aid in navigation.

With enough episodes, an RL agent can be expected to efficiently navigate their environment.

However, there are many types of RL agents and their classification determines how they navigate and learn their environment. The two types of RL agents that are appropriate for this study are q learning agents and SARSA agents.

Both agents utilize an internal q table that associates a q value for each state action pair. In this environment, a higher q value indicates a more favored action. The q values are updated differently for both of the agents. Q learning agents are off-policy agents that navigate an environment by balancing exploration and exploitation in a controlled exponential decay proportion and update the q table based on next maximum q value. For instance, if a q learned agent is at state 2 and traverses to state 3, the q value associated with the state action pair required to traverse between state 2 and state 3 would be updated by the maximum q value that can be achieved after state 3. This optimistic approach allows the agent to prioritize future gains over current rewards by assuming that the high q value will be exploited in the later training episodes. SARSA agents operate on an on-policy approach. Similar to q learning agents, they navigate the environment by balancing exploitation and exploration, but they do not update the q values of each state action pair in accordance with the highest plausible q value. They update them with the actual next q value. Stepping back into the previously instantiated example, if a SARSA agent traverses to state 3 from state 2 and then traverses to state 4 from state 3, the q value for the state action pair required to transition from state 2 to state 3 would factor in the q value of the transition between state 3 and state 4. This approach allows for a more realistic

metric where the agent only updates the q table based on actions that were already done—even during the exploration phase.

Both agents navigate the gold-rush environment. This environment is an 8 x 8 grid with empty spaces, mountains, mines, gold coins, and motherloads. The explorer shall be controlled by the agent. The explorer can transition to all empty spaces within 1 square from it in the ordinal direction of north, east, south, and west. If the agent attempts to transition the explorer to a location containing a mountain or attempts to move the explorer off the grid, the explorer returns to the same state. If the explorer attempts to transition to a location with a mine, the explorer loses, and the episode terminates. If the explorer attempts to transition to a location with gold coins, the explorer is rewarded but the episode is not terminated. This action transforms the state space into a new 8 x 8 grid with each location retaining its original object with the exception that the gold coins are gone. Finally, if the explorer transitions to the motherlode, the agent wins, and the episode is terminated. Each step that the explorer takes—any transition between states—results in a negative reward. This includes steps where the explorer returns to the original state, such as attempting to traverse into a mountain space or going off the grid. This is to penalize the agent from endlessly navigating the environment.

With both agents and the environment defined, we need to formulate a hypothesis on which type of agent is expected to perform better. Performance can be represented by the agents reward after completing an episode. The reward is the cumulation of all rewards incurred by the agent when traversing the state space from start to termination. Since q learning updates the q table based on the maximum potential of the next states while SARSA updates with the realistic potential of the next state, we can claim that q learning may result in quicker convergence than SARSA but have higher variability than SARSA. We also believe that exploring different starts may aid in slowing

down convergence and thus result in slightly less performance at the end of the training cycle for both q learning and SARSA.

METHODS

To fully evaluate our hypothesis, we created two types of learning agents: the q learner and the SARSA agents. Both agents were given the environment and were initialized with an empty dictionary. The dictionary served as the data structure holding the q table. The exploration rate was also initialized to 1—forcing the agents to perform exploration tactics 100% of the time.

After initialization of the q table, the agents were tasked to start at the starting position, state 0 if not exploring starts and random if so, and navigate the grid until the game terminated. The agents accomplished this task by first determining if they were exploring or exploiting. If they were exploring, they would select a random action. If they were exploiting, they would select the action that had the largest q value in accordance to what was stored within the q table—hence operating via an ϵ greedy approach. After selecting the action, the environment would update the state of the agent and return the new state, reward and a boolean indicating whether the game had terminated. The new state and the reward were saved and the agents would again select a new action. This cycle continues until the game is terminated. Once the game terminates, the agents move on to updating the q table. They examine the stored states, actions and their respective rewards and update the q value for each state action pair. The q learner updates each q value via the following equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_a Q(s', a') - Q(s, a) \right]$$

The SARSA learner updates each q value via:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$$

After the q table is updated with the respective q values, the cumulative reward for the episode is stored and the exploration rate is exponentially decayed with respect to the episode count. Once a certain number of episodes are completed, the agents terminate their traversals and return a list of the cumulative rewards of each episode.

Fifty of such q learning agents and fifty of such SARSA agents are created and trained and each of their cumulative reward lists are stored. The mean reward for each episode among the 50 agents is calculated and plotted. The standard error of the rewards is likewise calculated and plotted. Since the final fifty episodes of each agent consist of the agent navigating with a 0.01 exploration rate—only exploring 1% of the time, we pull out those 50 episodes and calculate their average reward. This serves as the $\bar{G}$ for the type of training agent. We also calculate the median for the final fifty episodes to reduce the impact of outliers in our data. This is determined as $\tilde{G}$.

RESULTS

Below are plots depicting the average reward per episode among all agents for each learning type, $\overline{G}_n$:

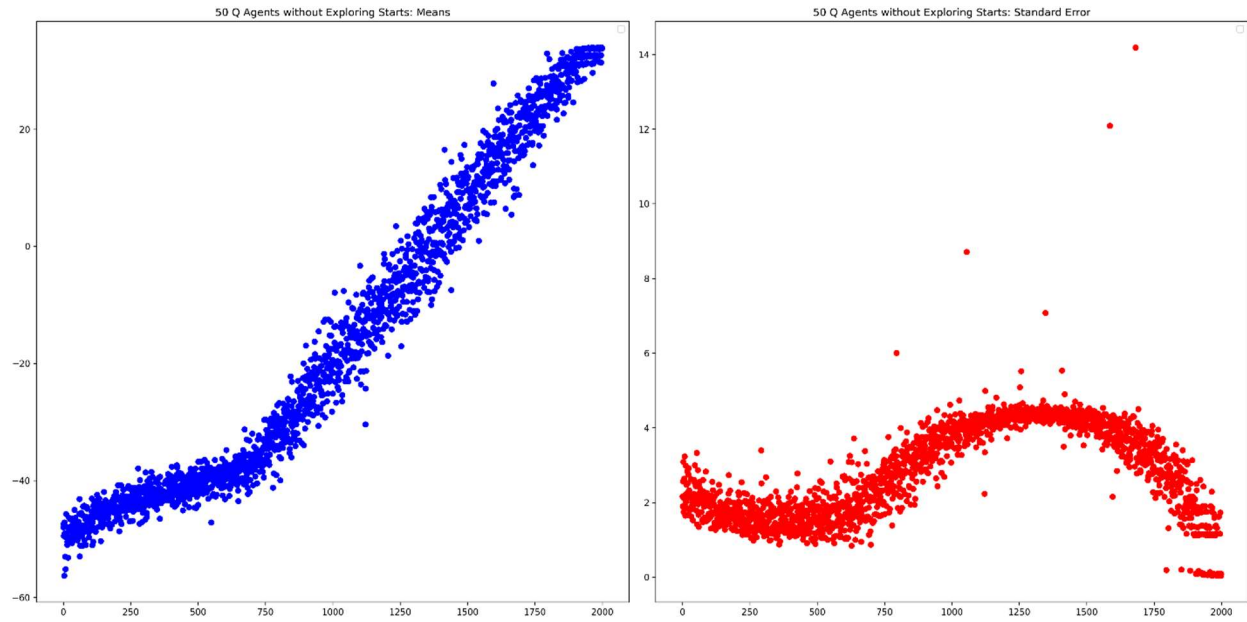


Figure 1: Q Learning Agents Mean per Episode (blue) and Standard Error per Episode (red)

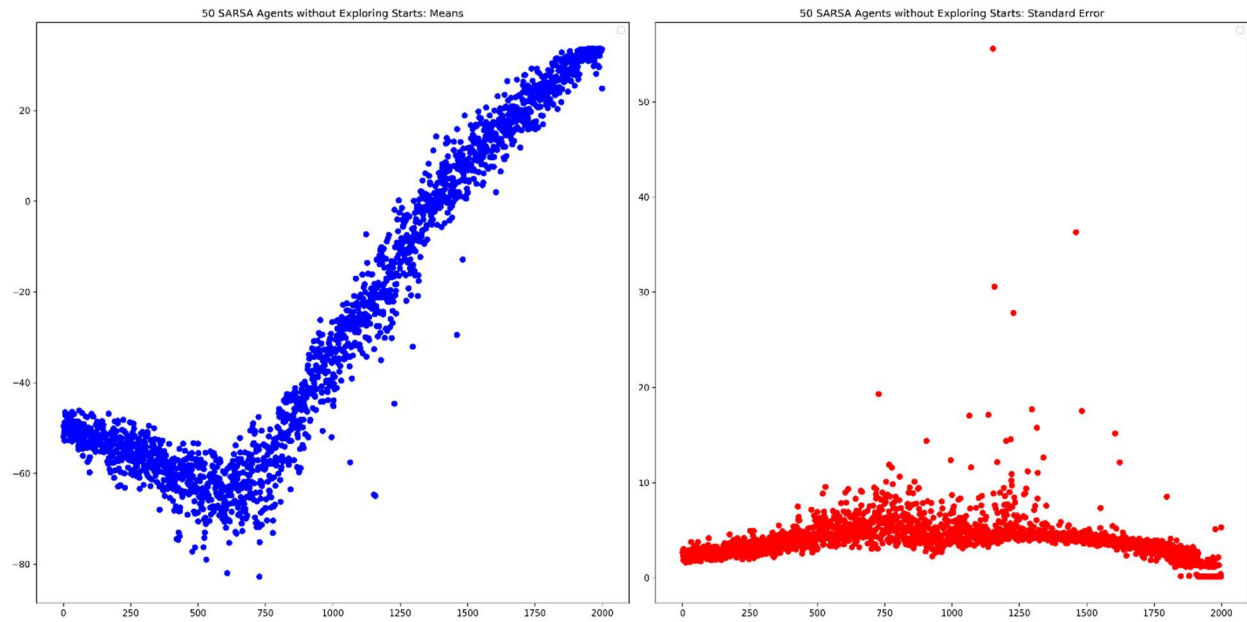


Figure 2: SARSA Learning Agents Mean per Episode (blue) and Standard Error per Episode (red)

Below are graphs containing the last 50 episodes for both learning types of learning agents:

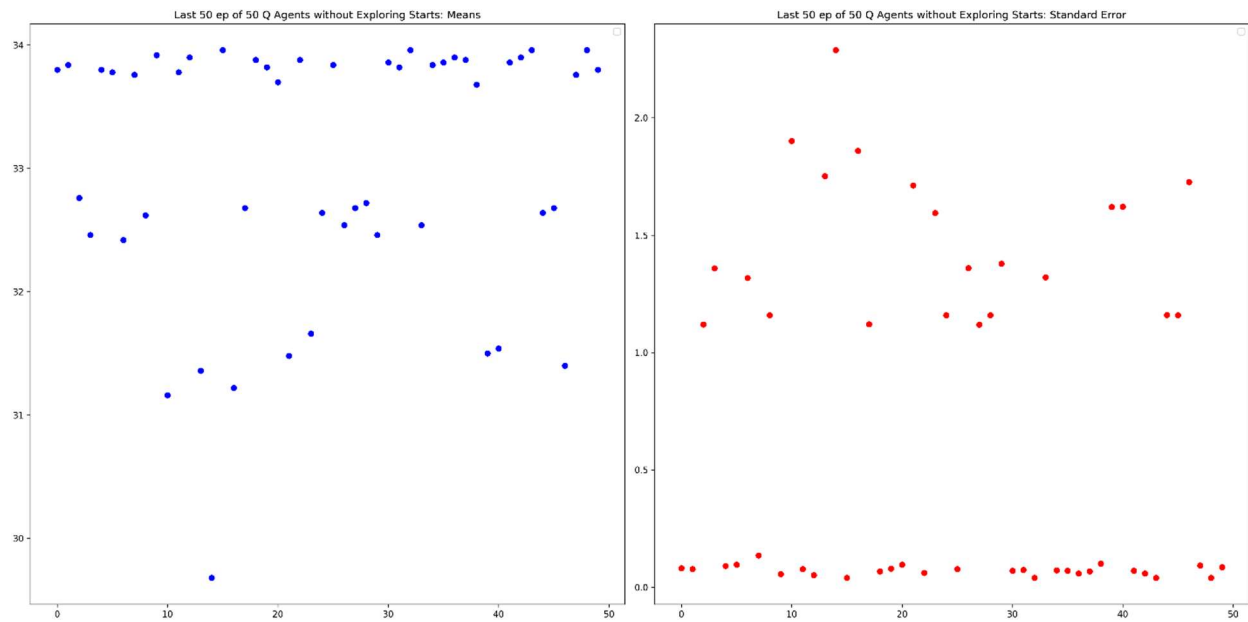


Figure 3: Q Learning Agents Mean per Episode (blue) and Standard Error per Episode (red) for the last 50 episodes

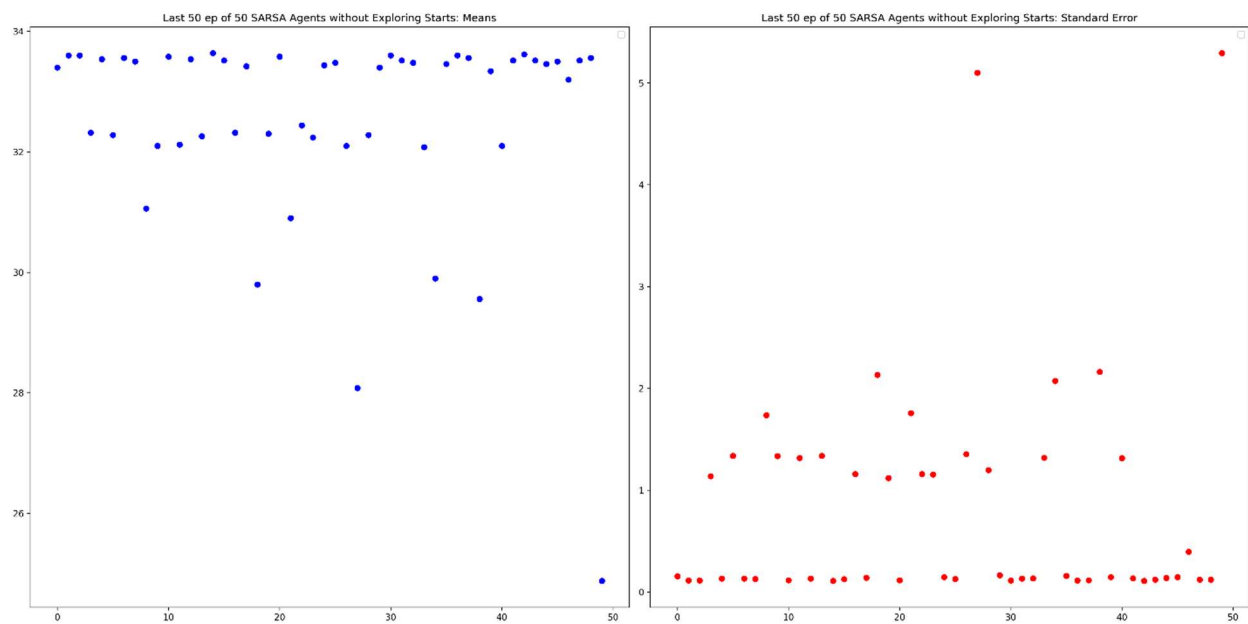


Figure 4: SARSA Learning Agents Mean per Episode (blue) and Standard Error per Episode (red) for the last 50 episodes

The previous graphs were performances where the agents were all initialized at the starting position. The next graphs add an option known as ‘exploring starts’. This randomizes the agent’s starting position. The performance of agents with this option enabled are shown below:

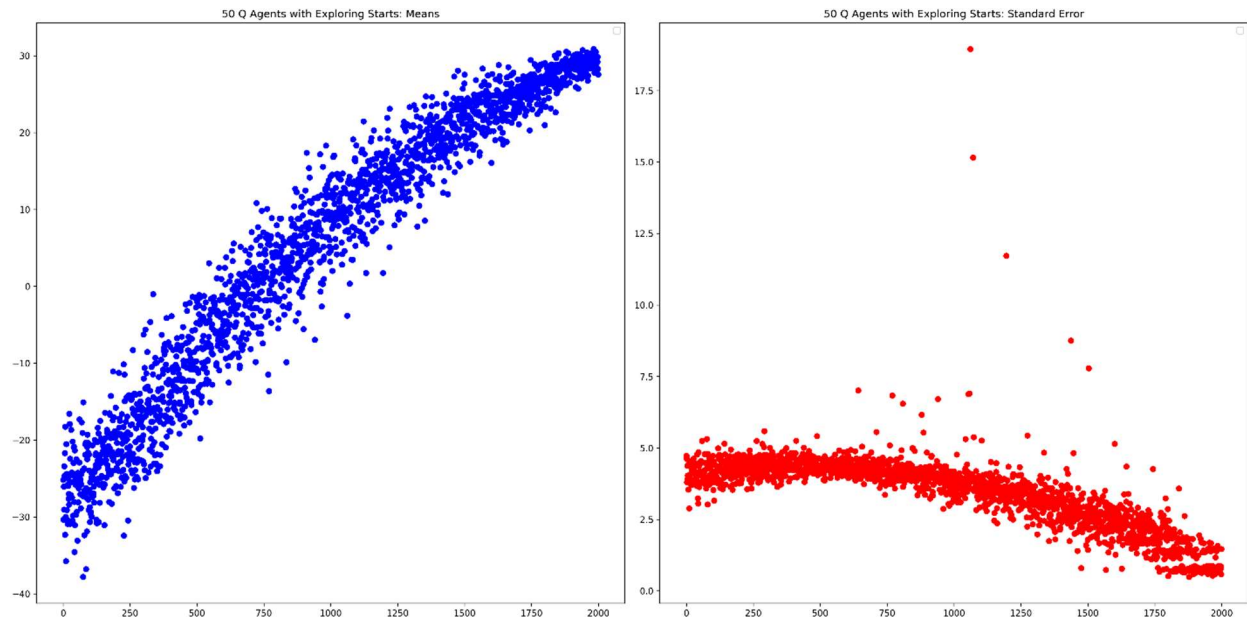


Figure 5: Q Learning Agents Mean per Episode (blue) and Standard Error per Episode (red)

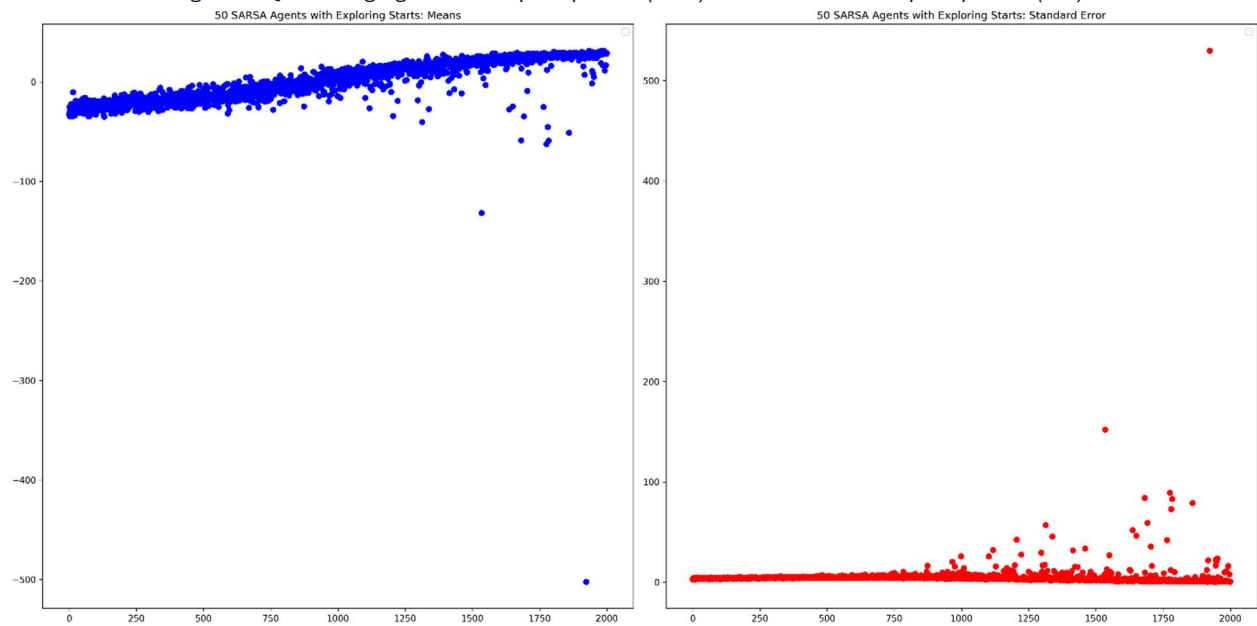


Figure 6: SARSA Learning Agents Mean per Episode (blue) and Standard Error per Episode (red)

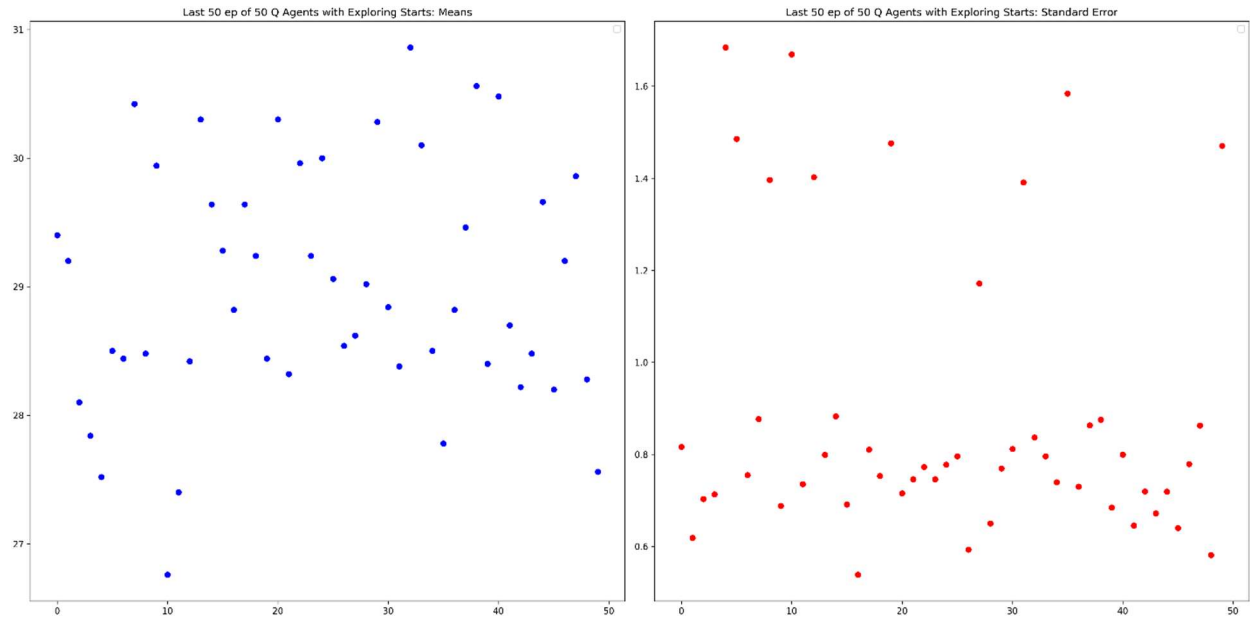


Figure 7: Q Learning Agents Mean per Episode (blue) and Standard Error per Episode (red) for the last 50 episodes

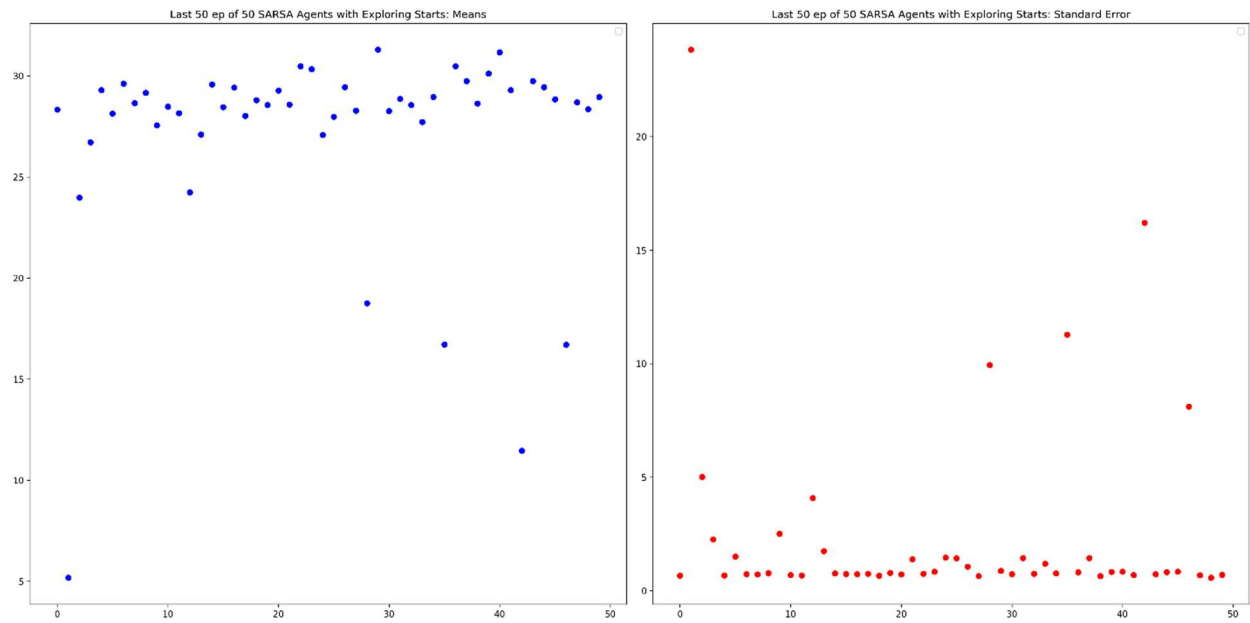


Figure 8: SARSA Learning Agents Mean per Episode (blue) and Standard Error per Episode (red) for the last 50 episodes

Below are the $\bar{G}$ and $\tilde{G}$ for all the learning agents:

Learning Agent	Exploring Starts?	$\bar{G}$	$\tilde{G}$
Q Agent	No	33.0508	33.76
Q Agent	Yes	28.9892	28.83
SARSA	No	32.5676	33.43
SARSA	Yes	27.1944	28.61

DISCUSSION

For the purposes of convenience, we shall restate our original hypothesis. Before conducting this study, we expected the q learning agents to converge more quickly than the SARSA agent but with slightly higher variability. We also expected that learning agents trained on exploring starts would converge slower and therefore have a slightly lower performance than their counter parts that were not trained via exploration of starts.

To address the first part of the hypothesis that the q learning agent would converge quicker than the SARSA, we can refer to the differences spotted in the mean plots of figure 1 and figure 2. Within the first couple episodes, both agents seem to be mainly exploring and their performances do not have defined slopes. However, there is a clear trend before the 750th episode. The q learning agent's performance increases slightly over the first 750 episodes which indicate that it's q table is being populated with some helpful values. The SARSA agent, on the other hand, sees a sizeable dip in it's performance over the first 750 episodes and this may be due to the q table being updated during the exploratory stage. This results in state action pairs having pessimistic values. After the 750th episode, both learning agents performances drastically increase at their

own rates. The q learning agents increase linearly until they get capped at a reward of around 33.76 while the SARSA agents increase logarithmically until they glide to a similar cap of 33.43. Due to the nature of the logarithmic function, it could be argued that at some point in the training, the SARSA agents were learning quicker than the q agents but at other points, the q learning agents outpaced the SARSA agents. What can be concluded for this information is that although SARSA can potentially learn quicker than q learning, the initial dip and logarithmic learning curve offset the SARSA agents far enough to allow for the q learning agents to converge to an optimal solution quicker.

When analyzing the standard error of the two graphs, we see that the q learning agents mimic a sinusoidal standard error graph that averages around three with some outliers in the 12+ range while the SARSA agents have a much more zoomed out graph with most error values hovering around four and some outliers in the 35+ range. Although both standard errors have similarly low error values and both standard errors seem to drop off near the end of the 2000 training episodes, SARSA demonstrates much more extreme outliers—to the contrast of our hypothesis. This could be due to the exponential decay hyperparameter that forced the SARSA agents to exploit quicker or slower than they were meant to resulting in wrongly calculated values. Upon debugging, it was also found that the SARSA agents would oscillate between a cycle of states, and this would easily ramp up the penalty of the episode. This may be due to the similar q values of multiple states and how easily they link back to one another. A way to resolve this is to incorporate some cycle detection logic to break the cycle via forceful exploration. Another way to resolve this is to

add a 'visit' count to each state which adds as an additional penalizing factor to the agent's q value.

When viewing figures 5 and 6, the performance of both agent learning types can be gleaned when trained with exploring starts. From these plots, it becomes clear that the q learning agent had higher average error than the q learning agents trained without exploring starts. However, the mean performance plot is a logarithmic curve that doesn't sport the initial slowly-increasing performance that the other q learning agents had. This proves that although exploring starts may result in slower convergence, the agents are able maintain a consistent speed of learning throughout the training period. The SARSA agents demonstrate extreme outliers due to the previously mentioned 'stuck agents' phenomenon. This resulted in skewing both the performance and the error data heavily. However, the SARSA agents begin to show higher variability near their final episodes much more when exploring starts than when not. This is in accordance with our hypothesis that agents trained via exploring starts will converge slower than those with a fixed start. It is worth noting that the performance of the SARSA agents seems to dwindle during the exploitation heavy episodes and this may hint at an improperly set decay parameter or some other hyperparameter.

To conclude, both q learning and SARSA learning agents can be applied to navigate this environment efficiently. Due to the similarity of the $\bar{G}$ and $\tilde{G}$ between the q learning and SARSA agents, we do not have sufficient evidence to prove that one performs better than the other. However, we were able to successfully prove that q learning converges slightly quicker than SARSA and boasts a slightly lower standard error for each episode. This may be due to agents getting stuck within the environment by cycling through a set of states. We also concluded that agents trained on exploring starts resulted in slower convergence and higher error than their

fixed-starting state counterparts. This explains the defined difference in $\bar{G}$ and $\tilde{G}$ of any learning agent and its exploring-starts equivalent.